```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt

print("--- 1. Data Preprocessing & Sequence Creation ---")
try:
    data = pd.read_csv('spg.csv')
except FileNotFoundError:
    print("ERROR: Please ensure 'spg.csv' is in the same directory.")
```

```
--- 1. Data Preprocessing & Sequence Creation ---
```

```
TARGET_COLUMN = 'generated_power_kw'
FEATURES_TO_DROP = [
    'total_precipitation_sfc',
    'snowfall_amount_sfc',
]
```

```
df_lstm = data.drop(columns=FEATURES_TO_DROP, errors='ignore')

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df_lstm)

def create_sequences(data, time_steps):
    X, y = [], []
    for i in range(len(data) - time_steps):
        X.append(data[i:(i + time_steps), :])

        y.append(data[i + time_steps, -1])
    return np.array(X), np.array(y)
```

```
TIME_STEPS = 4

X, y = create_sequences(scaled_data, TIME_STEPS)

TRAIN_SIZE = int(len(X) * 0.8)
X_train, X_test = X[:TRAIN_SIZE], X[TRAIN_SIZE:]
y_train, y_test = y[:TRAIN_SIZE], y[TRAIN_SIZE:]

print(f"LSTM Input Shape: {X_train.shape} (Samples, Time Steps, Features)")
print(f"Training samples after sequencing: {len(X_train)}")
print(f"Testing samples after sequencing: {len(X_test)}")
```

```
LSTM Input Shape: (3367, 4, 19) (Samples, Time Steps, Features)
Training samples after sequencing: 3367
Testing samples after sequencing: 842
```

```
print("\n--- 2. Building LSTM Network ---")

NUM_FEATURES = X.shape[2]

model = Sequential()
model.add(LSTM(
    units=50,
    return_sequences=True,
    input_shape=(TIME_STEPS, NUM_FEATURES)
))
model.add(Dropout(0.2))

model.add(LSTM(units=50))
model.add(Dropout(0.2))

model.add(Dense(units=1))

model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')

model.summary()
```

```
--- 2. Building LSTM Network ---
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_di
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 4, 50)	14,000
dropout (Dropout)	(None, 4, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Total params: 34,251 (133.79 KB)
Trainable params: 34,251 (133.79 KB)
Non-trainable params: 0 (0.00 B)

```
print("\n--- 3. Training and Evaluation ---")

history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

y_pred_scaled = model.predict(X_test)

dummy_scaler = MinMaxScaler(feature_range=(0, 1))
dummy_scaler.min_, dummy_scaler.scale_ = scaler.min_[-1], scaler.scale_[-1]

y_test_original = dummy_scaler.inverse_transform(y_test.reshape(-1, 1))
y_pred_original = dummy_scaler.inverse_transform(y_pred_scaled)

lstm_rmse = np.sqrt(mean_squared_error(y_test_original, y_pred_original))

print(f"Final LSTM Model RMSE: {lstm_rmse:.2f} kW")
```

```
--- 3. Training and Evaluation ---
Epoch 1/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 2s 23ms/step - loss: 0.0123 - val_loss: 0.0152
Epoch 2/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0122 - val_loss: 0.0147
Epoch 3/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0119 - val_loss: 0.0147
Epoch 4/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0121 - val_loss: 0.0140
Epoch 5/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0123 - val_loss: 0.0141
Epoch 6/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.0115 - val_loss: 0.0144
Epoch 7/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.0118 - val_loss: 0.0136
Epoch 8/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0112 - val_loss: 0.0141
Epoch 9/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0126 - val_loss: 0.0145
Epoch 10/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0116 - val_loss: 0.0143
Epoch 11/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0119 - val_loss: 0.0148
Epoch 12/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0121 - val_loss: 0.0145
Epoch 13/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0117 - val_loss: 0.0157
Epoch 14/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0113 - val_loss: 0.0148
Epoch 15/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0116 - val_loss: 0.0148
Epoch 16/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0113 - val_loss: 0.0144
Epoch 17/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0108 - val_loss: 0.0143
Epoch 18/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0112 - val_loss: 0.0142
Epoch 19/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0113 - val_loss: 0.0143
Epoch 20/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0106 - val_loss: 0.0152
```

```
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
Final LSTM Model RMSE: 429.43 kW
```

```
print("\n--- 4. Visualization ---")

plt.figure(figsize=(10, 4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('LSTM Model Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.tight_layout()
plt.savefig('W3_training_loss.png')
plt.close()
```

```
--- 4. Visualization ---
```

```
plt.figure(figsize=(12, 6))
plot_range = slice(100, 300)
plt.plot(y_test_original[plot_range], label='Actual Power (kW)', color='blue')
plt.plot(y_pred_original[plot_range], label='Predicted Power (kW)', color='red', linestyle='--')
plt.title(f'LSTM Prediction vs. Actual Power (Zoomed - RMSE: {lstm_rmse:.2f} kW)')
plt.xlabel('Time Step')
plt.ylabel('Generated Power (kW)')
plt.legend()
plt.tight_layout()
plt.savefig('W3_prediction_vs_actual.png')
plt.close()

print(f"\n*** Final LSTM RMSE is {lstm_rmse:.2f} kW ***")
```

```
*** Final LSTM RMSE is 429.43 kW ***
```